Ex: No: 10 Develop a Program to create a reliable TCP connection using Python Sockets.

Aim:

To design, implement, and verify a TCP socket-based reliable connection demonstration in Python using a listener and a connector running on both

Prerequisites:

Basic understanding of client-server architecture and TCP communication.

Knowledge of Python input/output operations and function calls.

Basic understanding of IP addresses, ports and sockets

Python installed and command prompt available on the system.

Algorithm (System Procedure):

Listener Program Procedure (listener.py):

1. Import the Socket module
2. Set the host address to 127.0.0.1 and port
3. Create a TCP socket using socket.AF_INET and socket.SOCK_STREAM.
4. Bind the socket to the localhost address and port
5. Listen for one incoming TCP connection.
6. Accept the connection and display the connector address.
7. Receive a message from the connector and display it.
8. Send a reply message back to the connector.
9. Close the connection and server socket.

Connected Program Procedure (Connector.py):

1. Import the socket module.
2. Set the host address to 127.0.0.1 and port to 5000
3. Create a TCP socket
4. Connect to the listener at the specified localhost address and port.
5. Send a hallwless test message to the listener.
6. Receive the reply from the listener.
7. Display the reply and close the socket.

Program Code:

Listener code: listener.py

```
import socket
HOST = "127.0.0.1"
PORT = 5000
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen()
print("Waiting for reverse connection...")
conn, addr = server.accept()
print("connected from:", addr)
message = conn.recv(1024).decode()
print("Received:", message)
conn.send("Hello from listener".encode())
conn.close()
server.close()
```

Connector code: connector.py

```
import socket
HOST = "127.0.0.1"
PORT = 5000
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
print("Connecting to listener...")
client.connect((HOST, PORT))
```

client.send("Hello from reverse connector")
reply = client.recv(1024).decode()
print("Listener replied:", reply)
client.close()

Execution steps & output:

Step 1: Start the listener process
Open command prompt in the experiment folder and run:

```
python listener.py
```

Step 2: Start the connector process
Open a second command prompt in the same experiment folder and run:

```
python connector.py
```

Final Output:

Listener Terminal output:

```
Waiting for reverse connection
Connected from: ('127.0.0.1', 54781)
Received: Hello from reverse connector
```

Connector Terminal output:

```
connecting to listener....
Listener replied: Hello from listener
```

Result:

The TCP socket based reverse connection demonstration was successfully executed.